

Algorithms

500+ One-Liners for Rapid Revision

IBPS SO (IT Officer) Examination - Complete Preparation Series

100% Syllabus Coverage - MCQ-Oriented - Zero Filler

Compiled by Asabhya Maanav

Table of Contents

Asymptotic Analysis & Growth of Functions	5
Notation Basics	5
Growth Ordering	5
Properties	5
Types of Analysis	6
Cases and Resources	6
Amortized Analysis	6
Recurrence Relations	6
Forming and Solving	6
Master Theorem	6
Searching	7
Linear Search	7
Binary Search	7
Other Searches	7
Comparison-Based Sorting	7
Bubble Sort	7
Selection Sort	8
Insertion Sort	8
Merge Sort	8
Quick Sort	8
Heap Sort	8
Shell Sort	8
Properties and Lower Bound	8
Non-Comparison Sorting	9
Counting / Radix / Bucket	9
Sorting Selection & Order Statistics	9
Hashing	9
Tables and Functions	9
Collision Resolution	9
Load Factor and Rehashing	10

Divide and Conquer	10
Greedy Algorithms	10
Dynamic Programming	10
Concepts	10
Classic Problems	10
Backtracking	11
Graph Basics	11
Graph Traversals	11
BFS and DFS	11
Applications	11
Minimum Spanning Tree	12
Shortest Path	12
Complexity Theory	12
Exam Focus Quick Hits	12
Extended Drill - Asymptotics & Recurrences	13
Extended Drill - Searching Nuances	13
Extended Drill - Sorting Internals	13
Extended Drill - Non-Comparison & Selection	14
Extended Drill - Hashing Details	14
Extended Drill - Greedy vs DP	15
Extended Drill - DP Recurrences	15
Extended Drill - Graphs	15
Extended Drill - MST & Shortest Path Tracing	16
Extended Drill - Complexity Classes & Misc	16
Extended Drill - Final Rapid Fire	16

Supplementary Drill - Deeper Numeric Facts	17
Supplementary Drill - Data Structure Tie-ins	17
Supplementary Drill - Algorithm Identification	17
Supplementary Drill - Stability & Properties Recap	18
Supplementary Drill - Graph Algorithm Specifics	18
Supplementary Drill - Master Theorem Practice	18
Supplementary Drill - Common Exam Pitfalls	18
Final Drill - Complexity Tables Recall	19
Final Drill - Space Complexity Recall	19
Final Drill - Paradigm Classification	19
Final Drill - True Facts Rapid Fire	20
Closing Drill - Last Mile Facts	20

Asymptotic Analysis & Growth of Functions

Notation Basics

- There are **3** primary asymptotic notations: **Big-O** (upper bound), **Big-Omega** (lower bound), and **Big-Theta** (tight bound).
- There are **2** strict notations: **little-o** (strictly slower) and **little-omega** (strictly faster).
- Big-O is defined as $0 \leq f(n) \leq c \cdot g(n)$ for $n \geq n_0$, giving the **upper bound**.
- Big-Omega is defined as $0 \leq c \cdot g(n) \leq f(n)$, giving the **lower bound**.
- Big-Theta requires **both** O and Omega: $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$, the **tight bound**.
- The mnemonic for the 5 relations is **O is $\leq$, o is $<$, Omega is $\geq$, omega is $>$, Theta is $=$** .
- Asymptotic analysis ignores **2** things: **constant factors** and **lower-order terms**.
- The purpose of asymptotic analysis is **machine-independence** and **input-representation independence**.
- For $3n^2 + 5n + 7$ the asymptotic class is **$O(n^2)$** , keeping only the **dominant term**.
- Little-o means **$\lim f/g = 0$** ; little-omega means **$\lim f/g = \text{infinity}$** .
- Big-O of the best case is **valid**; bounds (O/Omega/Theta) and cases (best/avg/worst) are **independent concepts**.

Growth Ordering

- The standard growth order is **$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$** , totaling **9** common classes.
- The **8** named complexity classes are **constant, logarithmic, linear, linearithmic, quadratic, cubic, exponential, factorial**.
- **Constant $O(1)$** is exemplified by **array index access** and **hash lookup (average)**.
- **Logarithmic $O(\log n)$** is exemplified by **binary search**.
- **Linearithmic $O(n \log n)$** is exemplified by **merge sort** and **heap sort**.
- **Quadratic $O(n^2)$** is exemplified by **bubble, selection, and insertion sort**.
- **Cubic $O(n^3)$** is exemplified by **Floyd-Warshall** and **naive matrix multiplication**.
- **Exponential $O(2^n)$** is exemplified by **subset enumeration** and **naive TSP**.
- **Factorial $O(n!)$** is exemplified by **permutation generation** and **brute-force TSP**.
- The base of a logarithm is **irrelevant** asymptotically because bases differ by a **constant factor**.
- **$\log(n!) = \Theta(n \log n)$** by **Stirling's approximation**, a frequent exam identity.
- **$\log(n^n) = n \log n$** , an identity that catches many students.
- **$\sqrt{n} = n^{0.5}$** sits between **$\log n$** and **n** in growth.
- To compare two functions, evaluate **$\lim f(n)/g(n)$** : **$0 \rightarrow o$, $\text{infinity} \rightarrow \text{omega}$, $\text{constant} \rightarrow \text{Theta}$** .
- Any **polynomial** dominates any **logarithm**; any **exponential** dominates any **polynomial**.

Properties

- Asymptotic notation has **transitivity**: $f=O(g)$, $g=O(h)$ implies **$f=O(h)$** .
- The sum rule states **$O(f)+O(g) = O(\max(f,g))$** .
- The product rule states **$O(f) \cdot O(g) = O(f \cdot g)$** .
- **Reflexivity** holds for O, Omega, Theta but **not** for little-o or little-omega.
- Transpose symmetry: **$f=O(g)$ iff $g=O(\text{Omega}(f))$** .
- **$2^{(n+1)} = O(2^n)$** (factor 2), but **$2^{(2n)} = 4^n$ is NOT $O(2^n)$** .

Types of Analysis

Cases and Resources

- There are **3** input cases analyzed: **best**, **average**, and **worst** case.
- An unqualified "complexity" in an MCQ usually means the **worst case**.
- **Worst case** gives a **guarantee**; **average case** assumes an **input distribution**.
- The **2** resources measured are **time complexity** (operations) and **space complexity** (extra memory).
- **Auxiliary space** is the extra memory **beyond the input**.
- In-place algorithms use **$O(1)$** or **$O(\log n)$** auxiliary space.
- **Insertion sort** best case **$O(n)$** occurs on **already-sorted** input.
- **Insertion sort** worst case **$O(n^2)$** occurs on **reverse-sorted** input.
- A loop **$i = i*2$** from 1 to n runs **$\log_2 n$** times.
- A loop **$i = i*3$** runs **$\log_3 n$** times.
- Two nested loops to n give **$O(n^2)$** ; a single loop gives **$O(n)$** .

Amortized Analysis

- **Amortized analysis** spreads the cost of expensive operations over a **sequence**, giving a worst-case-over-sequence guarantee.
- A **dynamic array** push (with doubling) is **$O(1)$ amortized** though a single resize is **$O(n)$** .
- There are **3** amortized methods: **aggregate**, **accounting (banker's)**, and **potential**.
- Amortized analysis differs from average case because it uses **no probability**.

Recurrence Relations

Forming and Solving

- The **3** recurrence-solving methods are **substitution**, **recursion tree**, and **Master theorem**.
- **Substitution method** works by **guessing** the form and proving by **induction**.
- **Recursion tree** sums **work per level** times **nodes per level**.
- **$T(n)=T(n-1)+c$** solves to **$O(n)$** (linear recursion).
- **$T(n)=T(n-1)+n$** solves to **$O(n^2)$** (insertion/selection sort).
- **$T(n)=2T(n-1)+1$** solves to **$O(2^n)$** (Tower of Hanoi).
- **$T(n)=T(n/2)+c$** solves to **$O(\log n)$** (binary search).
- **$T(n)=2T(n/2)+c$** solves to **$O(n)$** (tree traversal).
- **$T(n)=2T(n/2)+n$** solves to **$O(n \log n)$** (merge sort).
- **Tower of Hanoi** needs **$2^n - 1$** moves for n disks.

Master Theorem

- The Master Theorem solves **$T(n)=aT(n/b)+f(n)$** with **$a \geq 1, b > 1$** .
- The critical exponent is **$\log_b(a)$** , compared with **$f(n)$** .
- **Case 1**: f smaller by polynomial factor $\rightarrow$ **$\Theta(n^{\log_b a})$** (leaves dominate).
- **Case 2**: f equals $n^{\log_b a}$ $\rightarrow$ **$\Theta(n^{\log_b a} \cdot \log n)$** (balanced).
- **Case 3**: f larger by polynomial factor (with regularity) $\rightarrow$ **$\Theta(f(n))$** (root dominates).
- **$T(n)=2T(n/2)+n$** gives **$\Theta(n \log n)$** by case 2.

- $T(n)=4T(n/2)+n$ gives $\Theta(n^2)$ by case 1 ($\log_b a = 2$).
- $T(n)=2T(n/2)+1$ gives $\Theta(n)$ by case 1.
- $T(n)=T(n/2)+1$ gives $\Theta(\log n)$ by case 2.
- $T(n)=3T(n/2)+n$ gives $\Theta(n^{1.58})$ since $\log_2 3 \sim 1.58$.
- $T(n)=2T(n/2)+n^2$ gives $\Theta(n^2)$ by case 3.
- Master theorem **does not apply** when the gap between f and $n^{\log_b a}$ is **not polynomial** (e.g. $f = n \log n$).

Searching

Linear Search

- **Linear search** works on **unsorted** data with **no precondition**.
- Linear search best case is $O(1)$, average and worst are $O(n)$.
- Linear search worst-case comparisons = n ; average successful = $(n+1)/2$.
- Linear search uses $O(1)$ space.

Binary Search

- **Binary search** requires the array be **sorted** as a precondition.
- Binary search complexity is $O(\log_2 n)$ average/worst, $O(1)$ best.
- Safe mid computation is $\text{mid} = \text{low} + (\text{high}-\text{low})/2$ to avoid **overflow**.
- Maximum comparisons in binary search = $\text{floor}(\log_2 n) + 1$.
- Binary search recurrence is $T(n)=T(n/2)+O(1)$.
- Iterative binary search uses $O(1)$ space; recursive uses $O(\log n)$ stack.
- For **first occurrence**, on a match continue into the **left half**.
- For **last occurrence**, on a match continue into the **right half**.
- Count of a key = **last - first + 1** found in $O(\log n)$.

Other Searches

- **Interpolation search** averages $O(\log \log n)$ on **uniformly distributed** sorted data, worst $O(n)$.
- Interpolation probe formula is $\text{pos} = \text{low} + ((\text{key}-a[\text{low}])(\text{high}-\text{low}))/(\text{a}[\text{high}]-\text{a}[\text{low}])$.
- **Jump search** uses block size $\sqrt{n}$ giving $O(\sqrt{n})$ complexity.
- **Ternary search** splits into 3 parts giving $O(\log_3 n)$ levels but more comparisons than binary.
- To search an **unsorted** array fastest, use **linear search** (sorting first costs $O(n \log n)$).
- Binary search is **optimal** among comparison searches; ternary makes **more** comparisons.

Comparison-Based Sorting

Bubble Sort

- **Bubble sort** compares **adjacent** elements and needs up to $n-1$ passes.
- Bubble sort best case is $O(n)$ with **early-exit**, average/worst $O(n^2)$.
- Bubble sort worst comparisons and swaps = $n(n-1)/2$.
- Bubble sort is **stable**, **in-place**, and **adaptive** (with flag).
- The **early-exit optimization** stops when a pass makes **0 swaps**.
- After pass k of bubble sort, the largest k elements are in **final position**.

Selection Sort

- **Selection sort** selects the **minimum** of the unsorted part each pass.
- Selection sort is **$O(n^2)$** in **all** cases (best=avg=worst).
- Selection sort comparisons always = **$n(n-1)/2$** ; swaps = at most **$n-1$** .
- Selection sort is **NOT stable**, **not adaptive**, but **in-place**.
- Selection sort makes the **fewest swaps** $O(n)$ among simple sorts.

Insertion Sort

- **Insertion sort** builds a sorted prefix by **shifting** larger elements right.
- Insertion sort best case is **$O(n)$** (sorted), worst **$O(n^2)$** (reverse sorted).
- Insertion sort is **stable**, **in-place**, **adaptive**, and **online**.
- Insertion sort is best for **small** or **nearly-sorted** arrays.
- Insertion sort is used as the base case in hybrid sorts like **Timsort** and **Introsort**.

Merge Sort

- **Merge sort** is **divide-and-conquer** with a **merge** combine step.
- Merge sort is **$O(n \log n)$** in **all** cases (best=avg=worst).
- Merge sort recurrence is **$T(n)=2T(n/2)+n$** .
- Merge sort auxiliary space is **$O(n)$** ; it is **stable** but **not in-place**.
- Merge sort is preferred for **linked lists** and **external sorting**.

Quick Sort

- **Quick sort** picks a **pivot** and **partitions** elements around it.
- The **2** partition schemes are **Lomuto** (last pivot, more swaps) and **Hoare** (two pointers, fewer swaps).
- Quick sort is **$O(n \log n)$** best/average, **$O(n^2)$** worst.
- Quick sort worst case occurs on **sorted/reverse-sorted** input with bad pivot.
- Quick sort is **in-place**, **NOT stable**, with **$O(\log n)$** average stack space.
- Quick sort worst case is avoided via **randomized pivot** or **median-of-three**.
- Quick sort is the **fastest in practice** due to **cache locality** and low constants.

Heap Sort

- **Heap sort** builds a **max-heap** then extracts the max repeatedly.
- **Build-heap** is **$O(n)$** , and each extraction is **$O(\log n)$** , totaling **$O(n \log n)$** .
- Heap sort is **$O(n \log n)$** in all cases and uses **$O(1)$** space (**in-place**).
- Heap sort is **NOT stable** and has **poor cache locality**.
- Heap sort is the **in-place $O(n \log n)$ worst-case** sort.

Shell Sort

- **Shell sort** is a **generalized insertion sort** with decreasing **gaps**.
- Shell sort complexity ranges **$O(n^2)$** to **$O(n \log^2 n)$** by gap sequence.
- Shell sort is **in-place**, **NOT stable**, and **adaptive**.

Properties and Lower Bound

- The **5** stable sorts are **Bubble, Insertion, Merge, Counting, Radix** (mnemonic **BIM-CR**).
- The **4** unstable sorts are **Selection, Quick, Heap, Shell**.
- The comparison-sort lower bound is **$\Omega(n \log n)$** .
- The lower-bound proof uses a **decision tree** with **$n!$** leaves and height **$\log_2(n!)$** .
- Selection sort **minimizes writes/swaps** $O(n)$; bubble sort worst swaps = **$O(n^2)$** .

Non-Comparison Sorting

Counting / Radix / Bucket

- **Counting sort** runs in **$O(n+k)$** time and space for keys in range **$[0, k]$** .
- Counting sort is **stable** and efficient only when **$k = O(n)$** .
- **Radix sort** (LSD) runs in **$O(d(n+b))$** where d =digits, b =base.
- Radix sort requires its per-digit subroutine to be **stable** (usually counting sort).
- **LSD radix** processes the **least significant digit** first; MSD the most significant.
- **Bucket sort** averages **$O(n+k)$** for **uniformly distributed** input, worst **$O(n^2)$** .
- Linear-time sorts beat **$\Omega(n \log n)$** because they are **not comparison-based**.
- The **$\Omega(n \log n)$** lower bound applies **only** to **comparison sorts**.

Sorting Selection & Order Statistics

- The **k th order statistic** is the **k th smallest** element; the median is the **$n/2$ -th**.
- Finding k th element by sorting costs **$O(n \log n)$** ; by heap of size k costs **$O(n \log k)$** .
- **Median of medians** gives guaranteed **$O(n)$** worst-case selection.
- **QuickSelect** expected time is **$O(n)$** , worst **$O(n^2)$** .
- QuickSelect recurses into **only one side** giving **$n + n/2 + \dots = 2n$** .
- Finding **both min and max** needs only **$3 \cdot \text{floor}(n/2)$** comparisons.
- Choose **insertion sort** for small/nearly-sorted, **heap sort** for in-place guarantee, **merge sort** for stability.

Hashing

Tables and Functions

- A **hash table** gives average **$O(1)$** insert/search/delete.
- **Direct addressing** uses the key as index, only for **small integer universes**.
- The **division method** is **$h(k) = k \bmod m$** with m a **prime** away from powers of 2.
- The **multiplication method** is **$h(k) = \text{floor}(m \cdot (k \cdot A \bmod 1))$** , $A \sim 0.618$ (golden ratio).
- A good hash function is **deterministic, uniform, and fast**.

Collision Resolution

- The **2** broad collision strategies are **chaining** and **open addressing**.
- **Separate chaining** stores collisions in a **linked list** per slot.
- The **load factor** is **$\alpha = n/m$** (entries / slots).
- Chaining average search is **$O(1 + \alpha)$** and tolerates **$\alpha > 1$** .
- The **3** open-addressing probes are **linear, quadratic, and double hashing**.
- **Linear probing** is **$(h+i) \bmod m$** and causes **primary clustering**.

- **Quadratic probing** is $(h+c1.i+c2.i^2) \bmod m$ and causes **secondary clustering**.
- **Double hashing** is $(h1+i.h2) \bmod m$ and minimizes clustering.
- In double hashing, $h2(k)$ must be **relatively prime** to m .
- Open addressing requires **$\alpha \leq 1$** ; chaining does not.

Load Factor and Rehashing

- Open-addressing unsuccessful search probes $\sim 1/(1-\alpha)$.
- The typical rehash threshold is **$\alpha = 0.75$** (e.g. Java HashMap).
- **Rehashing** allocates a **larger (doubled)** table and reinserts keys, cost **$O(n)$** , amortized **$O(1)$** .
- Hash worst-case lookup is **$O(n)$** when all keys collide.
- Deletion in open addressing needs a **tombstone** marker.

Divide and Conquer

- The **3** D&C steps are **Divide**, **Conquer**, and **Combine**.
- D&C recurrences have the form **$T(n)=aT(n/b)+f(n)$** .
- **Binary search** is D&C with **no combine** step $\rightarrow O(\log n)$.
- **Merge sort** combine step is the **merge**, costing **$O(n)$** per level.
- **Maximum subarray** by D&C is **$O(n \log n)$** ; by **Kadane** it is **$O(n)$** .
- **Strassen's** matrix multiply uses **7** multiplications $\rightarrow O(n^{2.807})$.
- Strassen's exponent is **$\log_2 7 \sim 2.807$** , beating naive **$O(n^3)$** .
- Finding min and max together by D&C uses **$3 \cdot \text{floor}(n/2) \sim 1.5n$** comparisons.

Greedy Algorithms

- A **greedy algorithm** makes the **locally optimal** choice each step.
- Greedy needs **2** properties: **greedy-choice property** and **optimal substructure**.
- **Activity selection** sorts by **finish time** and selects greedily in **$O(n \log n)$** .
- **Fractional knapsack** takes items by highest **value/weight ratio** and greedy is **optimal**.
- **Huffman coding** merges the **two lowest**-frequency nodes to build an optimal **prefix code**.
- Huffman gives **shorter** codes to **more frequent** symbols, complexity **$O(n \log n)$** .
- **Job sequencing** sorts by **profit descending** and assigns latest free slot before deadline.
- **Greedy coin change** works only for **canonical** systems; fails for **[1,3,4]** making **6**.
- The **3** greedy graph algorithms are **Kruskal**, **Prim**, and **Dijkstra**.
- Greedy **fails** on **0/1 knapsack**, which needs **dynamic programming**.

Dynamic Programming

Concepts

- **Dynamic programming** needs **overlapping subproblems** and **optimal substructure**.
- The **2** DP styles are **memoization** (top-down) and **tabulation** (bottom-up).
- **Memoization** is recursive with a cache; **tabulation** is iterative bottom-up.
- DP differs from D&C because D&C subproblems are **independent (non-overlapping)**.

Classic Problems

- **Fibonacci** via DP is $O(n)$ vs $O(2^n)$ naive recursion.
- **Climbing stairs** has the **same recurrence** as Fibonacci.
- **0/1 Knapsack** DP is $O(nW)$, called **pseudo-polynomial**.
- **LCS** recurrence: match $\rightarrow dp[i-1][j-1]+1$, else $\max(dp[i-1][j], dp[i][j-1])$, complexity $O(mn)$.
- **LIS** is $O(n^2)$ by DP or $O(n \log n)$ with binary search.
- **Matrix chain multiplication** DP is $O(n^3)$ time, $O(n^2)$ space.
- Matrix chain **minimizes scalar multiplications**, not the actual product.
- **Edit distance** (Levenshtein) uses **insert/delete/replace**, complexity $O(mn)$.
- **Coin change** DP is $O(\text{amount} \times \text{coins})$ and handles non-canonical systems.
- **Subset sum** DP is $O(nT)$ and is **NP-complete** in general.
- **Rod cutting** DP is $O(n^2)$: $r(n) = \max(p[i] + r(n-i))$.
- **Min path sum** on a grid is $O(mn)$: $dp[i][j] = \text{cost} + \min(\text{top}, \text{left})$.
- **Floyd-Warshall** is DP over intermediate vertices, $O(V^3)$.

Backtracking

- **Backtracking** explores a **state-space tree** via **DFS** with **pruning**.
- **N-Queens** places n queens pruning on **row** and **diagonal** conflicts.
- The number of subsets is 2^n ; the number of permutations is $n!$.
- **Branch and bound** extends backtracking for **optimization** using **bounds**.
- Branch and bound is used for **0/1 knapsack** and **TSP** optimization.

Graph Basics

- **Adjacency matrix** uses $O(V^2)$ space with $O(1)$ edge lookup, best for **dense** graphs.
- **Adjacency list** uses $O(V+E)$ space with $O(\text{degree})$ lookup, best for **sparse** graphs.
- The **handshaking lemma** states sum of degrees = $2E$ in an undirected graph.
- The number of odd-degree vertices is always **even**.
- An undirected edge appears **twice** in an adjacency list.

Graph Traversals

BFS and DFS

- **BFS** uses a **FIFO queue** and runs in $O(V+E)$.
- **BFS** finds the **shortest path** (fewest edges) in **unweighted** graphs.
- **DFS** uses **recursion or a stack** and runs in $O(V+E)$.
- DFS classifies **4** edge types: **tree**, **back**, **forward**, **cross**.
- Undirected DFS has only **tree** and **back** edges.
- A **back edge** indicates a **cycle**.

Applications

- **Connected components** are found by running DFS/BFS from each unvisited node.
- **Undirected cycle** detection finds a **back edge** to a non-parent or union-find conflict.
- **Directed cycle** detection finds a back edge to a node in the **recursion stack**.
- **Bipartite check** uses **2-coloring**; a graph is bipartite iff it has **no odd-length cycle**.

- **Topological sort** orders a **DAG** so every edge goes **forward**.
- **Kahn's algorithm** repeatedly removes nodes of **in-degree 0**, in $O(V+E)$.
- DFS-based topological sort pushes nodes on **finish** and **reverses**.
- Topological sort exists **only for DAGs** (no cycles).
- **SCC** algorithms are **Kosaraju** (2 DFS) and **Tarjan** (1 DFS), both $O(V+E)$.

Minimum Spanning Tree

- A spanning tree of V vertices has exactly **$V-1$** edges.
- MST is unique when all edge weights are **distinct**.
- The **cut property** states the min-weight edge crossing any cut is in some MST.
- **Kruskal's** sorts edges ascending and uses **union-find**, complexity $O(E \log E)$.
- **Prim's** grows from a vertex adding the cheapest crossing edge.
- Prim with **adjacency matrix** is $O(V^2)$ (dense); with heap $O(E \log V)$.
- **Kruskal** is best for **sparse** graphs; **Prim** for **dense**.
- **Union-Find** supports **make-set, find, union**.
- **Path compression** + **union by rank** give $O(\alpha(n))$ per operation.

Shortest Path

- **BFS** solves unweighted shortest path in $O(V+E)$.
- **Dijkstra** requires **non-negative** weights; array $O(V^2)$, heap $O((V+E) \log V)$.
- **Dijkstra fails** on **negative edge weights**.
- **Bellman-Ford** handles **negative weights**, relaxing edges $V-1$ times, $O(VE)$.
- A **V -th** relaxation improvement in Bellman-Ford signals a **negative cycle**.
- **Floyd-Warshall** is **all-pairs**, $O(V^3)$ DP, handling **negative** edges.
- Floyd-Warshall detects a negative cycle if $\text{dist}[i][i] < 0$.
- **Dijkstra** is **greedy**; **Bellman-Ford** and **Floyd-Warshall** are **dynamic programming**.
- Path reconstruction uses a **predecessor/parent** array.

Complexity Theory

- **P** is solvable in **polynomial time**; **NP** is verifiable in polynomial time.
- **NP** means **nondeterministic polynomial**, NOT "non-polynomial".
- **NP-Complete** problems are the **hardest in NP** and all NP reduces to them.
- **NP-Hard** is at least as hard as NP-complete but **not necessarily in NP**.
- **NP-Complete = in NP AND NP-Hard**.
- The known relationship is **P subset of NP**; **P = NP** is **unproven**.
- **SAT** was the **first** proven NP-complete via the **Cook-Levin theorem**.
- Example NP-complete problems: **SAT, 3-SAT, TSP, vertex cover, clique, Hamiltonian cycle, subset sum, graph coloring**.
- Complexity classes (NP) are defined over **decision** (yes/no) problems.

Exam Focus Quick Hits

- Quicksort worst case is $O(n^2)$; merge and heap sort are always $O(n \log n)$.
- Build-heap is $O(n)$; heapify-down is $O(\log n)$.

- Stable sorts: **Bubble, Insertion, Merge, Counting, Radix**; in-place $O(1)$: **Bubble, Selection, Insertion, Heap**.
- Greedy works for **fractional knapsack**; DP needed for **0/1 knapsack**.
- Dijkstra fails on negatives; use **Bellman-Ford**.
- The asymptotic order is $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$.
- A spanning tree has **$V-1$** edges; handshaking sum of degrees = **$2E$** .
- Comparison sort lower bound is **$\Omega(n \log n)$** .

Extended Drill - Asymptotics & Recurrences

- The **3** conditions for $f=O(g)$ are constants **$c>0$** , **$n_0 \geq 0$** , and **$f(n) \leq c \cdot g(n)$** for $n \geq n_0$.
- **Theta** is the conjunction of **O** and **Omega**, the most precise of the **3** bounds.
- A function with different best and worst case (like quicksort) has **no single Theta** over all inputs.
- **n^{100}** grows slower than **2^n** eventually, proving polynomial loses to **exponential**.
- **$(\log n)^k$** (polylog) grows slower than **n^e** for any **$e>0$** .
- **$n!$** grows faster than **2^n** but slower than **n^n** , placing it **8th** of 9 classes.
- The sum **$1+2+\dots+n$** equals **$n(n+1)/2$** which is **Theta(n^2)**.
- The harmonic sum **$1+1/2+\dots+1/n$** equals **Theta($\log n$)**.
- A geometric series with ratio **>1** is dominated by its **last (largest)** term.
- A geometric series with ratio **<1** is dominated by its **first** term and is **$O(1)$** times it.
- **2 nested loops** where inner goes to i give total **$n(n+1)/2 = O(n^2)$** .
- A loop running while **$n>1$** , **$n=n/2$** executes **$\log_2 n$** iterations.
- The recurrence **$T(n)=T(\sqrt{n})+1$** solves to **$O(\log \log n)$** .
- The recurrence **$T(n)=2T(n/2)+n \log n$** solves to **$O(n \log^2 n)$** (gap case, not master).
- **Merge step** of merge sort does at most **$n-1$** comparisons per level.
- The **number of levels** in a balanced halving recursion is **$\log_2 n + 1$** .
- Big-O upper bound for linear search best case is still **$O(n)$** as an upper bound though it runs in **$O(1)$** .

Extended Drill - Searching Nuances

- Binary search on **n** elements needs at most **$\text{ceil}(\log_2(n+1))$** comparisons.
- Binary search divides the search space by **2** each step (factor **$1/2$**).
- For an array of **1000** elements binary search needs about **10** comparisons ($2^{10}=1024$).
- For **1,000,000** elements binary search needs about **20** comparisons.
- Interpolation search reduces remaining elements to about **$\sqrt{n}$** of previous on uniform data.
- **Exponential search** finds a range by doubling, then binary-searches, in **$O(\log i)$** for position i .
- Linear search is the **only** option for **unsorted linked lists** without extra structure.
- Binary search needs **random access**, so it suits **arrays** not plain linked lists.
- The midpoint formula **$(\text{low}+\text{high})/2$** risks overflow when **$\text{low}+\text{high} > \text{INT_MAX}$** .
- Ternary search evaluates **2** midpoints per iteration versus **1** for binary.

Extended Drill - Sorting Internals

- Bubble sort performs exactly **$n-1$** comparisons in its **first** pass.
- After **k** passes of bubble sort, **$n-k$** elements remain unsorted.
- Optimized bubble sort detects a sorted array in **1** pass with **0** swaps.
- Selection sort performs **$n-1$** comparisons in the first pass and **1** in the last.

- Insertion sort on a sorted array does **$n-1$** comparisons and **0** shifts.
- Insertion sort on reverse-sorted data does **$n(n-1)/2$** comparisons and shifts.
- The number of **inversions** equals the number of swaps insertion sort performs.
- Merge sort always splits into halves of size **$\text{floor}(n/2)$** and **$\text{ceil}(n/2)$** .
- Quicksort's partition places the pivot in its **final sorted position**.
- **Lomuto partition** maintains one boundary index; **Hoare** uses two converging pointers.
- A **good pivot** (median) makes quicksort **$O(n \log n)$** ; the worst pivot makes it **$O(n^2)$** .
- **Randomized quicksort** expected time is **$O(n \log n)$** regardless of input order.
- Heap sort first **builds the heap in $O(n)$** then does **$n-1$** extractions of $O(\log n)$.
- A binary max-heap stores the maximum at the **root (index 0 or 1)**.
- In a heap, node at index i has children at **$2i+1$** and **$2i+2$** (0-based).
- In a heap, the parent of index i is at **$\text{floor}((i-1)/2)$** (0-based).
- A heap of n elements has height **$\text{floor}(\log_2 n)$** .
- The **last non-leaf node** of a heap is at index **$n/2 - 1$** (0-based).
- **Heapify (sift-down)** costs **$O(\log n)$** ; building the heap bottom-up costs **$O(n)$** .
- Shell sort with **gap=1** reduces to plain **insertion sort**.
- Among simple $O(n^2)$ sorts, **insertion** is fastest on nearly-sorted data.
- **Merge sort** is the standard choice for **stable + guaranteed $O(n \log n)$** .
- **Heap sort** is the standard choice for **$O(n \log n) + O(1)$ space**.
- **Timsort** (Python/Java) is a hybrid of **merge sort** and **insertion sort**.
- **Introsort** (C++ STL) combines **quicksort, heapsort, and insertion sort**.

Extended Drill - Non-Comparison & Selection

- Counting sort is **not suitable** when the key range **k** greatly exceeds **n** .
- Counting sort builds a **cumulative/prefix-sum** array to find output positions.
- Radix sort with **d** digits performs **d** stable counting-sort passes.
- Radix sort on b -bit integers with r -bit digits does **b/r** passes.
- Bucket sort works best when keys are **uniformly distributed** over a known range.
- Bucket sort degrades to **$O(n^2)$** when all elements land in **one bucket**.
- The median is the **$(n+1)/2$** -th order statistic for odd n .
- QuickSelect avoids sorting the **unneeded** partition, saving work.
- The **median-of-medians** algorithm guarantees a good pivot for **$O(n)$** worst-case selection.

Extended Drill - Hashing Details

- A **perfect hash** has **no collisions** for a fixed key set.
- **Universal hashing** picks a hash function **randomly** to bound expected collisions.
- The **birthday paradox** implies collisions appear after about **$\text{sqrt}(m)$** insertions.
- Linear probing's cluster grows because adjacent slots fill, worsening **primary clustering**.
- Quadratic probing with table size **prime** and $\alpha < 0.5$ guarantees an empty slot is found.
- Double hashing needs **$h_2(k) \neq 0$** and **coprime** with m to probe all slots.
- Chaining search cost is proportional to chain length **$1 + \alpha$** .
- For open addressing at **$\alpha=0.5$** , expected unsuccessful probes ~ 2 .
- For open addressing at **$\alpha=0.9$** , expected unsuccessful probes ~ 10 .
- A hash table with **m** slots and **n** keys has load factor **n/m** .

- Resizing typically **doubles** capacity to keep amortized insert **$O(1)$** .

Extended Drill - Greedy vs DP

- Greedy makes **one** irrevocable choice per step; DP **explores** choices via subproblems.
- Greedy proof needs **exchange argument** or **matroid** theory (awareness).
- Dijkstra** is greedy because it finalizes the **minimum-distance** frontier vertex.
- Prim and Kruskal** are greedy because they add the **minimum-weight** safe edge.
- Activity selection chooses **earliest finishing** compatible activity for the optimum.
- Huffman's tree has **frequent symbols near the root** (short codes).
- A Huffman code is a **prefix code**: no codeword is a prefix of another.
- 0/1 knapsack** fails greedy because a high-ratio item may **block** a better combination.
- Fractional knapsack allows **splitting** items, making greedy optimal.
- Coin change is greedy-safe for **canonical** denominations like **1,2,5,10**.

Extended Drill - DP Recurrences

- Fibonacci space can be reduced to **$O(1)$** keeping only the **last two** values.
- 0/1 knapsack space can be reduced to **$O(W)$** using a **1D rolling** array.
- LCS of two length- n strings is at most **n** and at least **0** .
- The LCS length relates to edit distance and the **longest common substring** (different problem).
- Edit distance between identical strings is **0** ; between disjoint length- m and length- n is **$\max(m,n)$** .
- LIS using patience sorting maintains **tails** and binary-searches, giving **$O(n \log n)$** .
- Matrix chain DP fills the table by increasing **chain length**.
- Multiplying matrices $A(p \times q)$ and $B(q \times r)$ costs **$p \times q \times r$** scalar multiplications.
- Coin change min-coins uses **$dp[a] = \min(dp[a - \text{coin}] + 1)$** .
- Coin change count-ways iterates coins **outer** to avoid counting permutations.
- Subset sum and **0/1 knapsack** share the same **pseudo-polynomial** DP structure.
- Rod cutting is equivalent to **unbounded knapsack** with lengths as weights.
- DP table for a grid of $m \times n$ cells uses **$O(mn)$** space, reducible to **$O(n)$** .

Extended Drill - Graphs

- A graph with V vertices can have at most **$V(V-1)/2$** edges if undirected simple.
- A directed simple graph can have at most **$V(V-1)$** edges.
- A **tree** on V nodes has exactly **$V-1$** edges and **no cycle**.
- A **complete graph** K_n has **$n(n-1)/2$** edges.
- A graph is **dense** if $E \sim V^2$, **sparse** if $E \sim V$.
- BFS** from a source labels vertices with their **level/distance**.
- DFS** discovery and finish times satisfy the **parenthesis theorem**.
- A graph is a **DAG** if DFS finds **no back edge**.
- A connected undirected graph with **$V-1$** edges is a **tree**.
- An undirected graph is **Eulerian** (circuit) iff all degrees are **even**.
- An undirected graph has an **Euler path** iff exactly **0 or 2** odd-degree vertices.
- Topological order** is not unique unless the DAG is a **total order (chain)**.
- Kahn's algorithm** detects a cycle if processed nodes **$< V$** .

Extended Drill - MST & Shortest Path Tracing

- Kruskal processes edges in **nondecreasing weight** order.
- Kruskal uses union-find **find** to test if endpoints are already connected.
- Prim's heap version decreases keys of **adjacent** vertices when a cheaper edge is found.
- Both Prim and Kruskal produce an MST of total weight that is **identical** (if unique).
- Dijkstra uses a **min-priority queue** keyed by tentative distance.
- Dijkstra **relaxes** edges of the extracted (finalized) vertex.
- Bellman-Ford performs **V-1** rounds of relaxing **all E** edges.
- Bellman-Ford on a graph with **negative cycle** reports it on the **V-th** round.
- Floyd-Warshall's core update is **$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$** .
- Floyd-Warshall's outer loop iterates over **intermediate vertex k**.
- Running Dijkstra from **all V** sources costs **$O(VE \log V)$** versus Floyd-Warshall **$O(V^3)$** .
- For sparse all-pairs, **Johnson's algorithm** beats Floyd-Warshall (awareness).
- Negative weights but no negative cycle: use **Bellman-Ford** or **Floyd-Warshall**.

Extended Drill - Complexity Classes & Misc

- **P** is contained in **NP**, which is contained in **PSPACE** (awareness).
- A problem is **NP-Hard** if every NP problem **reduces** to it in polynomial time.
- **Reductions** are the tool used to prove a problem **NP-Complete**.
- The **halting problem** is **undecidable**, harder than NP-Hard.
- **TSP optimization** is NP-Hard; its **decision** version is NP-Complete.
- **Vertex cover**, **clique**, and **independent set** are mutually related NP-Complete problems.
- If any NP-Complete problem is solved in **polynomial time**, then **P=NP**.
- **Approximation algorithms** tackle NP-Hard optimization with bounded error.
- **Decision problems** output **yes/no**; **optimization** outputs the best value.
- The **2** classic certificate ideas: NP solutions are **verifiable** quickly, not necessarily **findable** quickly.

Extended Drill - Final Rapid Fire

- Bubble, insertion, and selection sort all use **$O(1)$** auxiliary space.
- Merge sort needs **$O(n)$** auxiliary space, the most of the common sorts.
- The fastest comparison sort by worst-case guarantee that is in-place is **heap sort**.
- The fastest average-case practical sort is **quicksort**.
- The only common $O(n \log n)$ **stable** sort is **merge sort**.
- Counting, radix, and bucket sort can achieve **linear** time under assumptions.
- Binary search, merge sort, and quicksort are all **divide and conquer**.
- Kruskal, Prim, Dijkstra, Huffman, activity selection, and fractional knapsack are **greedy**.
- LCS, edit distance, matrix chain, 0/1 knapsack, coin change, and LIS are **dynamic programming**.
- N-Queens, subsets, and permutations are solved by **backtracking**.
- BFS uses **queue**; DFS uses **stack**; Dijkstra/Prim use **priority queue**; Kruskal uses **union-find**.
- The master theorem has **3** cases comparing $f(n)$ with $n^{\log_b a}$.
- A spanning tree always has **V-1** edges and connects all **V** vertices.
- The comparison-sorting lower bound **$\Omega(n \log n)$** comes from a **decision tree** of $n!$ leaves.
- Build-heap is **$O(n)$** , a frequent trap versus the wrong answer **$O(n \log n)$** .

- Quicksort worst case $O(n^2)$, merge/heap worst case $O(n \log n)$.
- Dijkstra needs **non-negative** weights; Bellman-Ford allows **negative**, Floyd-Warshall is **all-pairs**.
- Greedy gives the optimum for **fractional** knapsack; DP is needed for **0/1** knapsack.
- Hashing average lookup is $O(1)$; worst case is $O(n)$.
- NP means **nondeterministic polynomial**, defined for **decision** problems.

Supplementary Drill - Deeper Numeric Facts

- $\log_2$ of **1024** is **10**, of **1,048,576** is **20**, of **16** is **4**.
- $2^{10} = 1024$, $2^{16} = 65536$, $2^{20} \sim 1 \text{ million}$, $2^{30} \sim 1 \text{ billion}$.
- The number of nodes in a complete binary tree of height h is $2^{(h+1)}-1$.
- The number of leaves in a complete binary tree of height h is 2^h .
- A binary tree with n nodes has height between $\log_2 n$ (balanced) and $n-1$ (skewed).
- The number of comparisons to merge two sorted arrays of sizes m and n is at most $m+n-1$.
- Selecting 2 of n for pairwise min-max gives $\text{floor}(n/2)$ pairs.
- The factorial $5! = 120$, $6! = 720$, $10! = 3,628,800$.
- The number of binary search tree shapes on n keys is the **Catalan number** $C(n)$.
- The n th Catalan number is $(2n)! / ((n+1)! n!)$.
- **Bubble sort** is named for elements "bubbling" to their position; it is **rarely used** in practice.
- **Cocktail shaker sort** is a bidirectional **bubble sort** variant.
- **Gnome sort** resembles insertion sort using **single swaps**.
- **Cycle sort** minimizes the number of **writes** to memory.
- **Pancake sort** sorts using prefix **reversals** (flips).
- **Tim sort** has worst case $O(n \log n)$ and is **stable**.
- **Introsort** guarantees $O(n \log n)$ by switching quicksort to heapsort on deep recursion.

Supplementary Drill - Data Structure Tie-ins

- A **stack** is **LIFO**; DFS uses it implicitly via recursion.
- A **queue** is **FIFO**; BFS uses it for level-order traversal.
- A **priority queue** (heap) supports **extract-min/max** in $O(\log n)$.
- A binary heap supports **insert** and **delete** in $O(\log n)$, **peek** in $O(1)$.
- A **balanced BST** (AVL, Red-Black) supports search/insert/delete in $O(\log n)$.
- A **trie** supports string search in $O(\text{length})$ independent of dataset size.
- **Union-Find** with optimizations approaches $O(\alpha(n)) \sim$ effectively constant per op.
- **Adjacency list** of a graph supports neighbor iteration in $O(\text{degree})$.
- An **indexed/Fenwick tree (BIT)** supports prefix sums in $O(\log n)$.
- A **segment tree** supports range queries and updates in $O(\log n)$.

Supplementary Drill - Algorithm Identification

- An algorithm that halves its input each step is likely $O(\log n)$.
- An algorithm with two recursive calls on halves plus linear work is $O(n \log n)$.
- An algorithm with two recursive calls on $n-1$ is likely $O(2^n)$.
- A doubly nested loop both to n is $O(n^2)$; triply nested is $O(n^3)$.
- A single pass over input is $O(n)$; constant-time work is $O(1)$.

- Generating all subsets is $O(2^n)$; all permutations is $O(n!)$.
- Matrix multiplication of two $n \times n$ matrices naively is $O(n^3)$.
- All-pairs shortest path by Floyd-Warshall is $O(V^3)$.
- Sorting then scanning for duplicates is $O(n \log n)$; hashing makes it $O(n)$.
- BFS/DFS visiting every vertex and edge once is $O(V+E)$.

Supplementary Drill - Stability & Properties Recap

- **Bubble sort** is stable because it only swaps **adjacent** unequal-order elements.
- **Insertion sort** is stable because equal keys are **not shifted past** each other.
- **Merge sort** is stable when the merge prefers the **left** half on ties.
- **Selection sort** is unstable because a **long-distance swap** can reorder equal keys.
- **Quick sort** is unstable because partitioning **swaps far apart** elements.
- **Heap sort** is unstable because heap operations **reorder** equal keys.
- **Counting sort** is stable when filling output **right to left** using prefix sums.
- **Radix sort** relies on a **stable** digit sort to preserve earlier digit order.
- An **adaptive** sort runs faster on partially sorted input: **insertion** and **bubble (flagged)**.
- A **non-adaptive** sort ignores existing order: **selection sort**.

Supplementary Drill - Graph Algorithm Specifics

- **BFS** tree gives shortest unweighted paths from the **source**.
- **DFS** is used for **topological sort**, **SCC**, and **cycle detection**.
- A **bridge** is an edge whose removal **disconnects** the graph.
- An **articulation point** is a vertex whose removal **increases components**.
- **Tarjan's algorithm** finds bridges and articulation points in $O(V+E)$.
- A graph is **2-edge-connected** if it has **no bridges**.
- **Bipartite** graphs are exactly those with **no odd cycle**, 2-colorable.
- A **complete bipartite** graph $K(m,n)$ has $m \cdot n$ edges.
- **Topological sort** via Kahn uses a queue of **in-degree 0** vertices.
- The number of edges in any tree (MST included) is $V-1$.

Supplementary Drill - Master Theorem Practice

- $T(n)=8T(n/2)+n^2$ gives **$\Theta(n^3)$** ($\log_b a=3$, case 1).
- $T(n)=7T(n/2)+n^2$ (Strassen) gives **$\Theta(n^{\log_2 7}) \sim n^{2.807}$** (case 1).
- $T(n)=2T(n/2)+\log n$ gives **$\Theta(n)$** (case 1).
- $T(n)=16T(n/4)+n$ gives **$\Theta(n^2)$** ($\log_4 16=2$, case 1).
- $T(n)=2T(n/4)+1$ gives **$\Theta(\sqrt{n})$** ($\log_4 2=0.5$, case 1).
- $T(n)=2T(n/4)+\sqrt{n}$ gives **$\Theta(\sqrt{n} \log n)$** (case 2).
- $T(n)=3T(n/4)+n \log n$ gives **$\Theta(n \log n)$** (case 3).
- $T(n)=T(n/2)+n$ gives **$\Theta(n)$** (case 3, root dominates).

Supplementary Drill - Common Exam Pitfalls

- **Big-O is not always the worst case** - it is an upper bound regardless of case.

- **Build-heap is $O(n)$** , not $O(n \log n)$ - the most common heap trap.
- **Quicksort is not stable** despite being the most popular sort.
- **Merge sort is not in-place** despite being $O(n \log n)$ and stable.
- **Selection sort is not stable and not adaptive** but makes the fewest swaps.
- **Dijkstra fails on negative edges** even without a negative cycle.
- **Greedy fails on 0/1 knapsack** - the canonical greedy-vs-DP trap.
- **NP does not mean non-polynomial** - it is nondeterministic polynomial.
- **Spanning tree has $V-1$ edges**, not V edges.
- **$\log(n!) = \Theta(n \log n)$** , not $\Theta(n)$ or $\Theta(n^2)$.
- **Counting sort range k** can make it worse than $O(n \log n)$ if k is large.
- **Ternary search is slower** than binary search in comparisons.
- **Topological sort needs a DAG**; cycles break it.
- **Interpolation search worst case is $O(n)$** , not $O(\log \log n)$.
- **Quadratic probing** causes **secondary** clustering, linear causes **primary**.

Final Drill - Complexity Tables Recall

- Linear search is **$O(n)$** worst, binary search **$O(\log n)$** , interpolation **$O(\log \log n)$** average.
- Bubble/selection/insertion sorts are **$O(n^2)$** average; merge/heap **$O(n \log n)$** ; quick **$O(n \log n)$** average.
- Counting sort is **$O(n+k)$** , radix **$O(d(n+b))$** , bucket **$O(n+k)$** average.
- Hash insert/search/delete is **$O(1)$** average, **$O(n)$** worst.
- BFS and DFS are **$O(V+E)$** with adjacency lists.
- Dijkstra is **$O((V+E) \log V)$** with a binary heap.
- Bellman-Ford is **$O(VE)$** ; Floyd-Warshall is **$O(V^3)$** .
- Kruskal is **$O(E \log E)$** ; Prim is **$O(E \log V)$** or **$O(V^2)$** .
- 0/1 knapsack is **$O(nW)$** ; LCS/edit distance **$O(mn)$** ; matrix chain **$O(n^3)$** .
- QuickSelect is **$O(n)$** expected; median-of-medians **$O(n)$** worst.

Final Drill - Space Complexity Recall

- Bubble/selection/insertion/heap sort use **$O(1)$** auxiliary space.
- Merge sort uses **$O(n)$** ; quicksort uses **$O(\log n)$** stack on average.
- Counting sort uses **$O(n+k)$** ; radix **$O(n+b)$** ; bucket **$O(n+k)$** .
- Recursive binary search uses **$O(\log n)$** stack; iterative uses **$O(1)$** .
- BFS uses **$O(V)$** for the queue and visited array.
- DFS uses **$O(V)$** for the recursion stack worst case.
- Dijkstra/Prim use **$O(V)$** for the priority queue.
- DP tables for LCS/edit distance use **$O(mn)$** , reducible to **$O(\min(m,n))$** .
- Adjacency matrix uses **$O(V^2)$** ; adjacency list uses **$O(V+E)$** .
- Hash table uses **$O(n+m)$** space for n keys and m slots.

Final Drill - Paradigm Classification

- **Divide and conquer**: binary search, merge sort, quicksort, Strassen, max-subarray, min-max.
- **Greedy**: activity selection, fractional knapsack, Huffman, job sequencing, Kruskal, Prim, Dijkstra.

- **Dynamic programming**: Fibonacci, 0/1 knapsack, LCS, LIS, matrix chain, edit distance, coin change, subset sum, rod cutting, Floyd-Warshall, Bellman-Ford.
- **Backtracking**: N-Queens, subset generation, permutation generation, Sudoku.
- **Branch and bound**: 0/1 knapsack optimization, TSP optimization.
- The **4** major design paradigms are **divide-and-conquer**, **greedy**, **dynamic programming**, **backtracking**.
- **Bellman-Ford** is DP; **Dijkstra** is greedy - they solve the same single-source problem differently.
- **Floyd-Warshall** is DP over **intermediate vertices**, an all-pairs method.
- **Kadane's algorithm** is DP solving max-subarray in **$O(n)$** .
- **Memoization** converts exponential recursion to polynomial by **caching**.

Final Drill - True Facts Rapid Fire

- A min-heap's smallest element is at the **root**, retrievable in **$O(1)$** .
- A max-heap's largest element is at the **root**.
- Heapsort uses a **max-heap** for ascending order.
- The merge in merge sort is **stable** and **$O(n)$** per level.
- Binary search requires **$O(1)$** random access, ideal for **arrays**.
- Quicksort partition is **in-place** using **$O(1)$** extra space.
- Counting sort is **not** a comparison sort, so it beats **$\Omega(n \log n)$** .
- A DAG always has at least **one** vertex of in-degree **0**.
- A DAG always has at least **one** topological ordering.
- Dijkstra finalizes vertices in **nondecreasing** distance order.
- Bellman-Ford can be **terminated early** if a full pass makes **no relaxation**.
- Floyd-Warshall is simplest to code with **three nested loops**.
- Prim's matrix version is preferred when the graph is **dense** ($E \sim V^2$).
- Kruskal's edge-sort version is preferred when the graph is **sparse**.
- The MST and shortest-path tree are generally **different** trees.
- A negative-weight cycle makes shortest paths **undefined** (negative infinity).
- Hash collisions are **unavoidable** when keys exceed slots (pigeonhole).
- The pigeonhole principle guarantees a collision when **$n > m$** keys in m slots.
- A load factor near **1** drastically slows **open addressing**.
- Rehashing keeps amortized insertion at **$O(1)$** despite occasional **$O(n)$** resizes.

Closing Drill - Last Mile Facts

- The **3** cases of the master theorem compare $f(n)$ to **$n^{\log_b a}$** via polynomial gaps.
- A recursion tree has **$\log_b n$** levels with **a^i** nodes at level i .
- **Insertion sort** is the preferred sort for arrays smaller than about **10-20** elements.
- **Quicksort** median-of-three picks the median of **first, middle, last** as pivot.
- **Hoare partition** does fewer swaps but does not place the pivot at its **final** index.
- **Lomuto partition** always places the pivot at its **final** position.
- The **number of leaf nodes** in quicksort's recursion tree is **n** .
- **Merge sort** on a linked list needs only **$O(1)$** extra space (pointer rearrangement).
- **Counting sort** is the engine inside **radix sort** for each digit.
- **Bucket sort** typically uses **insertion sort** within each bucket.
- A **hash function** mapping to m slots should distribute keys **uniformly**.

- The **division hash** avoids m as a power of **2** to use all key bits.
- **Dijkstra** with a Fibonacci heap is $O(E + V \log V)$.
- **Prim** with a Fibonacci heap is $O(E + V \log V)$.
- The **decrease-key** operation drives the heap complexity of Dijkstra and Prim.
- **BFS** and **Dijkstra** coincide when all edge weights equal **1**.
- **Topological sort** can be used to find the **longest path** in a DAG in $O(V+E)$.
- The **transitive closure** of a graph is computable by **Floyd-Warshall**-style DP.
- **N-Queens** for $n=8$ has **92** distinct solutions.
- A **Hamiltonian path** visits every vertex once; deciding it is **NP-Complete**.
- An **Euler circuit** visits every edge once and needs all **even** degrees.
- The **traveling salesman problem** seeks the shortest **Hamiltonian cycle** and is **NP-Hard**.
- **Greedy** algorithms never **reconsider** earlier choices.
- **Dynamic programming** trades **memory for time** by caching subproblems.
- The **two** pillars of DP are **overlapping subproblems** and **optimal substructure**.
- **Amortized $O(1)$** dynamic-array growth uses **geometric** (doubling) resizing.